

《100天风控专家》

规则的D类调优(1): cutoff收紧

出品: 东哥起飞

解锁风控课程



关注我的公众号





1. D类调优的业务场景



扫码进专属群



D类调优 业务场景

监控发现逾期上升

通过监控报表发现，逾期率有明显上升的趋势，需要及时定位原因，调整策略。

实际逾期率超过预期目标

实际逾期率比离线分析时要高很多，且超过了预期的目标值，需要调整成，与目标一致。

放款头寸变少

机构放款资金不足时，可以收紧策略，只放款给资质较好的用户。

市场环境下行控制风险敞口

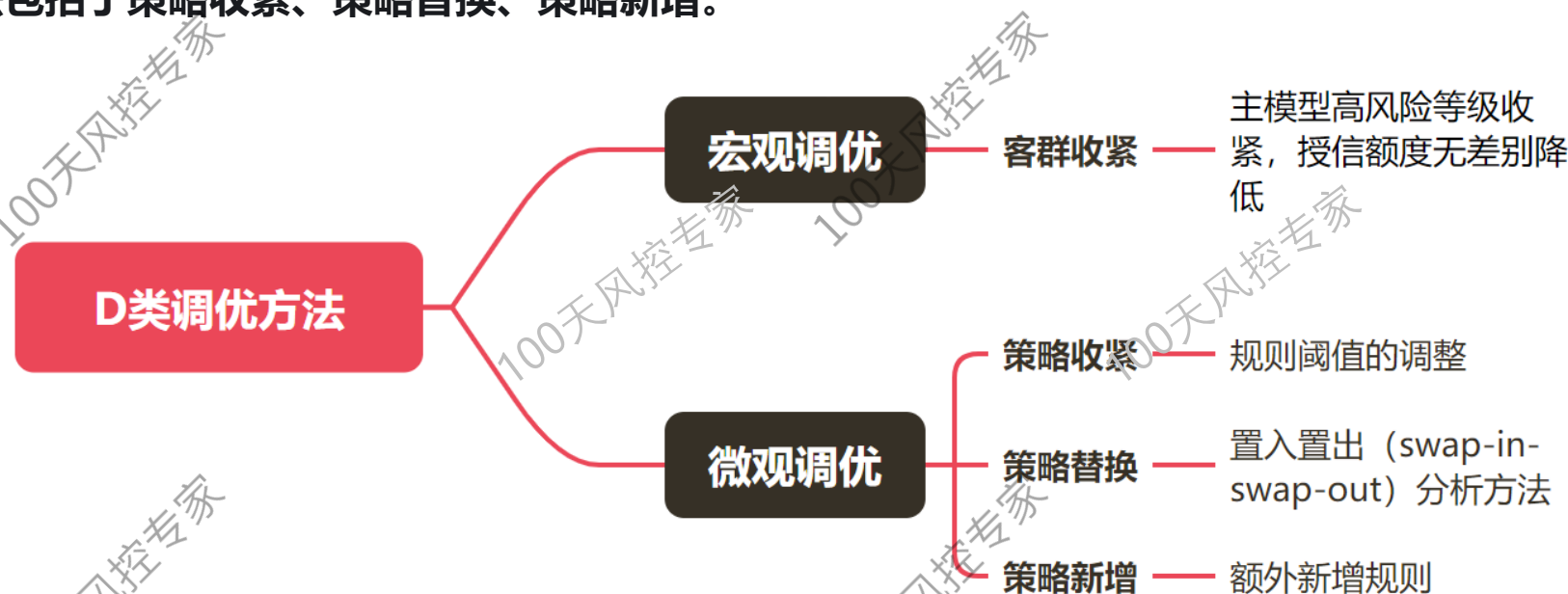
外部市场环境下行，资产质量整体变差，可以适当收紧策略，控制风险敞口。



2. D类调优的方法论

D类调优可分为宏观和微观两个层面的。

- ✓ **宏观层面**：对“整体风险客群”的收紧，而非某个具体维度的客群；
- ✓ **微观层面**：对某个维度的客群（非整体）进行策略收紧，以达到优化策略效用的目的，**具体方法**包括了策略收紧、策略替换、策略新增。



扫码进专属群





3. 什么是策略收紧?

1) 概念理解

策略收紧（狭义上理解）这里特指，对规则或模型的阈值进行收紧调整，将原通过的客群进行一定比例的拒绝，以达到降低逾期率的目的。注：这里的策略收紧指的是狭义上的理解，一种具体的方法，仅对规则或模型本身的阈值进行调整，不借助其他变量（比如策略新增、策略替换）。

2) 适用对象

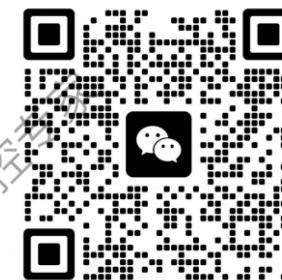
策略收紧的分析方法同时适用于规则和模型（因为模型本身就是一个强规则）。规则是微观的调整，调整范围小，而模型是宏观调整，收紧的调整范围更大，比如对主模型融合模型的风险等级进行收紧拒绝一类风险客群，那将会大幅度的降低通过率和逾期率。

3) 方法特点

策略收紧是通过有贷后表现的样本分析后制定的，和A类调优的阈值放松相比，有更好的量化分析条件基础，分析过程更容易。



扫码进专属群





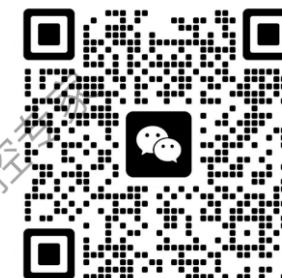
4. 阈值收紧的分析步骤

无论是对规则还是模型，其阈值收紧的分析步骤大致如下：

- 1) **调优需求**：政策指导、市场变化导致资产质量变差、监控报表发现逾期率升高等
- 2) **提取样本**：挑选合适的历史样本（代表性、充分性、数据可回溯性、时效性），并且回溯数据（具体可参考模型篇“样本设计”内容）
- 3) **寻找风险**：一般遵循“从大到小”的顺序向下拆解逐个分析和排查出风险点（寻找可优化点）
- 4) **量化分析**：对分析对象(规则或模型)进行分箱，统计分箱下的样本数量、区间坏账率（bad rate）和Lift值等评估指标，确定可调优的对象。
- 5) **制定策略**：执行策略调整方案(可能有多个)，对比分箱下bad rate和大盘风险指标，制定阈值收紧策略
- 6) **效果测试**：在“历史样本、近期样本”上执行收紧策略，对通过率、逾期率进行效果测算，与收紧前对比。如效果未达预期需重新调整策略后再进行评估，直到满足要求为止，完成最终调整方案的确定。
- 7) **策略上线**：决策引擎配置策略后，进行灰度测试或者AB测试，最后再正式上线



扫码进专属群



5. 量化分析

1) 背景介绍

策略分析人员通过监控报表发现，近期FPD7+的首逾指标不断升高，达到风险预警线需要进行策略调整。假设经过分析发现了贷前策略的“规则集”在通过样本上的风险较高，此时我们需要对该规则集内部的规则进行量化分析。

对规则集内部所有的单个规则进行分箱处理，并计算分箱下的区间坏账率 bad rate 和 Lift，对比找出 Lift 值较大的，结合业务逻辑来确定可调优的规则对象。比如下面是其中一个多头类的规则（含义是“最近6个月申请贷款机构数”），它在线上已有的判断逻辑是： **≥ 4 时命中拒绝，否则未命中通过**。分析后发现已通过样本的 [3,4) 区间对应的 $Lift=3.87>3$ ，在所有规则中最高（bad rate接近 31%，远高于大盘平均风险水平7.95%）。

	variable	bin	count	count_distr	good	bad	badprob	woe	bin_iv	total_iv	breaks	is_special_values	lift
0	ovd_order_cnt_6m_grade	[-inf,2.0)	37512	0.829049	35837	1675	0.044652	-0.613362	0.241951	0.793710	2.0	False	0.561999
1	ovd_order_cnt_6m_grade	[2.0,3.0)	4663	0.103057	3689	974	0.208878	1.118106	0.203903	0.793710	3.0	False	2.628963
2	ovd_order_cnt_6m_grade	[3.0,int)	3072	0.067894	2126	946	0.307943	1.640050	0.347857	0.793710	inf	False	3.875795



扫码进专属群





6. 制定策略方案

对于“规则阈值收紧”的调整比较简单，比如上面的多头类规则，就是将其阈值 ≥ 4 拒绝变为 ≥ 3 拒绝（以下红色部分）。

	variable	bin	count	count_distr	good	bad	badprob	woe	bin_iv	total_iv	breaks	is_special_values	lift
0	ovd_order_cnt_6m_grade	[-inf,2.0)	37512	0.829049	35837	1675	0.044652	-0.613362	0.241951	0.793710	2.0	False	0.561999
1	ovd_order_cnt_6m_grade	[2.0,3.0)	4663	0.103057	3689	974	0.208878	1.118106	0.203903	0.793710	3.0	False	2.628963
2	ovd_order_cnt_6m_grade	[3.0,inf)	3072	0.067894	2126	946	0.307943	1.640050	0.347857	0.793710	inf	False	3.875795

在一个规则集内部，可调整规则对象可能不止一个，需决定对哪些规则进行调整。一般的经验方法是：

- 1) 筛选规则变量 bad rate 有单调性的，且具有一定的业务含义
- 2) 然后优先筛选 Lift 值较大的（单调性较强的），在信贷场景中一般 Lift 2-3 可认为提升度较大（至少为2以上），可进行收紧处理。因此需对所有变量Lift值排序，然后挑选符合标准的来做收紧

3) 在制定策略时由于不清楚调整后的效果，可先制定出一个初版的调整方案做效果测算，如不满足要求，再返回进行规则组合的增减。此外还需注意规则阈值调整后对未调整其他规则的影响，可分析各规则的单一命中率的变化。



扫码进专属群





6. 制定策略方案

制定策略优化的方案是一个分析的过程，确定好之后需将策略用代码在当前离线分析环境下执行。

右侧的函数ruleset_calc为执行策略的Python代码，将调整后的策略执行后计算规则集的**综合命中率**、**单一命中率**、**自然命中率**，主要用于反映规则集命中的变化情况、以及内部规则互相之间的影响。

```
def ruleset_calc(df:pd.DataFrame, lst:list):
    """
    规则集计算分析函数
    """
    data = df.copy()
    # 创建规则命中变量，默认值为0
    for n in lst:
        data[str(n[0])+'_hit'] = 0

    # 循环所有规则，计算命中情况
    for m in lst:
        if m[2] == 'lt':
            cond = data[str(m[0])] < m[1]
            data[str(m[0])+'_hit'] = data[str(m[0])+'_hit'].mask(cond, other=1)
        else:
            cond = data[str(m[0])] > m[1]
            data[str(m[0])+'_hit'] = data[str(m[0])+'_hit'].mask(cond, other=1)

    # 计算综合命中率
    rule_hit = [c for c in data.columns if '_hit' in c]
    data['hit_any'] = data[rule_hit].any(axis=1)
    print('##### 综合命中率 #####')
    print('综合命中率为:{}'.format(data['hit_any'].mean()*100))

    # 计算规则的单一和自然命中率
    data['hit_sum'] = data[rule_hit].sum(axis=1)
    print('##### 规则命中数量分布 #####')
    display(data['hit_sum'].value_counts(normalize=True))

    pure_hit_info = []
    for col in rule_hit:
        hit_rate = round(data[data[col]==1].shape[0]*100/data.shape[0],4)
        pure_hit_sum = data[(data[col]==1)&(data['hit_sum']==1)].shape[0]
        pure_hit_rate = round(pure_hit_sum*100/data.shape[0],4)
        if hit_rate == 0:
            hit_rate = 0.00001
        pure_hit_pct = round(pure_hit_rate*100/hit_rate,4)
        pure_hit_info.append((col, pure_hit_rate, hit_rate, pure_hit_pct))

    pure_hit_df = pd.DataFrame(pure_hit_info, columns=['rule','pure_hit_rate','hit_rate','pure_hit_pct'])
    pure_hit_df = pure_hit_df.sort_values(['pure_hit_rate','pure_hit_pct'],ascending=[False,False])
    print('##### 单一/自然命中率 #####')
    display(pure_hit_df.style.background_gradient(cmap="Greens", subset=['pure_hit_rate','pure_hit_pct']))

    return data['hit_any'].mean(),data
```



扫码进专属群





7. 效果测算

效果测试主要是评估，调整前后策略对于“**通过率、逾期率**”的变化影响。理论上来说，做规则阈值收紧的D类调优后，通过率和逾期率会同步下降，如何去评估调优后的效果呢？

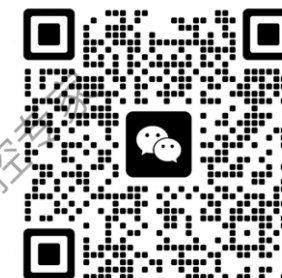
可以“**对比通过率和逾期率下降的幅度**”来评估是否有效。因为通过率和逾期率的量纲不一致，对比绝对值是没有意义的，因此可对比相对值。比如收紧调整后，逾期率下降幅度为20%，通过下降幅度为6.79%，说明逾期率下降的程度更多，即牺牲了少部分好客户拒绝了更多的坏客户。

此外还要注意，如果是日常策略调整（微调），业务上不允许大幅度的下降通过率，这会直接导致业务不稳定，一般可控制在1-2%左右或者不变；如果是做大规模的收紧调整，通过率下降幅度较大的情况，则需要更详细的效果测算。

比如，可以从“**成本收益**”的角度进行测算。按照“**其他成本(资金成本、人力成本、投放成本、运营成本、数据成本等)+风险损失成本>=利息+罚息**”的公式，如果策略收紧调整后，增加拒绝的客群中，成本总和超过了收益总和，则认为策略是有效的。不过该测算过程需要额外补充和匹配还款相关的数据，另外其他成本项也需要进行合理的预估。



扫码进专属群



谢谢

出品人：东哥起飞

解锁风控课程



关注我的公众号



Python数据科学



扫码进专属群

